GeoInspect

GIS-enabled field inspection assignment platform

A four-service application that prevents double-booking of field inspectors, makes assignment creation idempotent, and remains usable when notifications fail.

Course	Scalable Services (SEZG583)
Assignment	Assignment 1 - Design and Development of a Microservices Application
Repository	https://github.com/sharmaark/SS_Assignment
Team	[Add student names and IDs before submission]
Prepared	2026-10-13

Rubric traceability

Criterion	Marks	What the evaluator can verify	Report location
Problem and decomposition	2	Meaningful GIS field-inspection problem; capability boundaries; clear ownership; rejected alternatives	Pages 2-3
Architecture and API design	2	Protocol-labelled diagrams, communication choice, OpenAPI, error contract, explicit states	Pages 4-8
Implementation quality	3	Runnable services, rules, validation, environment config, tests, repository hygiene	Pages 2, 8, 11
Engineering challenges	3	Concurrency, concurrent idempotency, and resilience with repeatable scripts and limits	Pages 9-10 and 6
Report, video and evidence	3	Claim-to-file mapping, editable diagrams, live-demo plan, submission links	Pages 11-12

Submission note

The report describes the code and evidence currently present in the repository. Team details, individual reflections, and the final Google Drive/video link are deliberately left as labelled fields because they require student-owned information.

1. Problem, scope and business rules

Municipal and utility organisations create field work orders at geographic locations and assign inspectors for a time slot. The scarce resource is an inspector's availability. Two coordinators can request the same inspector and slot concurrently; a naive availability check can approve both. Retried client requests can also create duplicate assignments, while notification providers may respond slowly, fail, or commit a message before the HTTP response is lost.

Design objective: complete a traceable three-service assignment transaction while preserving inspector capacity and making retries safe. Notification is the deliberately unreliable fourth service.

Business rules and traceability

Rule	Enforced behavior	Code location	Unit test
BR1	The inspector's rectangular service area must cover the inspection point; boundaries count as covered.	services/workforce/src/business.py	services/workforce/tests/test_business.py
BR2	The inspector must have the work order's required skill.	services/workforce/src/business.py	services/workforce/tests/test_business.py
BR3	An inspector may hold only one reservation for a time slot.	services/workforce/src/business.py + DB UNIQUE constraint	services/workforce/tests/test_business.py
BR4	Only the assignment that owns a work-order claim may confirm it.	services/work_order/src/business.py	services/work_order/tests/test_business.py
BR5	Reusing an Idempotency-Key with a different payload is rejected with HTTP 409.	services/dispatch/src/business.py	services/dispatch/tests/test_business.py
BR6	JPEG/PNG photos within the size limit may be attached only after assignment.	services/work_order/src/business.py	services/work_order/tests/test_business.py

Scope assumptions

- Dispatch is the workflow orchestrator. Calls use REST over HTTP and JSON; every service owns its persistence.
- A work order stores a point location and optional inspection photos. MongoDB GridFS stores photo bytes and metadata; files are not embedded in API payloads.
- The demo uses a single slot string as the contention key. Production scheduling could use intervals and geospatial travel constraints.

Out of scope

Authentication, UI, route optimisation, offline mobile synchronisation, real SMS/email, and full image-processing pipelines are excluded by design. This keeps the one-week implementation focused on the assignment's distributed-system requirements.

2. Service decomposition and ownership

The boundaries follow business capabilities that can change independently. Each service can be explained in one sentence and has its own dependency file, environment template, API layer, business layer, data layer, database, health endpoint, logs, and OpenAPI document.

Service	Responsibility	Owns	Why this boundary
Work-order :8001	Create field jobs, validate GIS points, control job state, store photos	MongoDB work orders; GridFS files	Location/photo lifecycle differs from staff scheduling and uses document/file storage.
Workforce :8002	Register inspectors and reserve a slot atomically	PostgreSQL inspectors and reservations	Capacity is a consistency boundary that benefits from a relational UNIQUE constraint.
Dispatch :8003	Orchestrate assignment and store idempotent outcomes	PostgreSQL assignments and idempotency records	Workflow state and replay semantics belong together; it does not own inspectors or work orders.
Notification :8004	Simulate slow/failing provider and deduplicate notifications	MongoDB notification attempts/results	External failure behavior can evolve independently and is isolated from the core transaction.

Database-per-service ownership

Data	Authoritative service	Referenced elsewhere as
Work order and GIS point	Work-order	work_order_id
Photo file and metadata	Work-order / GridFS	photo_id
Inspector	Workforce	inspector_id
Slot reservation	Workforce	reservation_id
Assignment + replay response	Dispatch	assignment_id / Idempotency-Key
Notification	Notification	notification_id and logical key

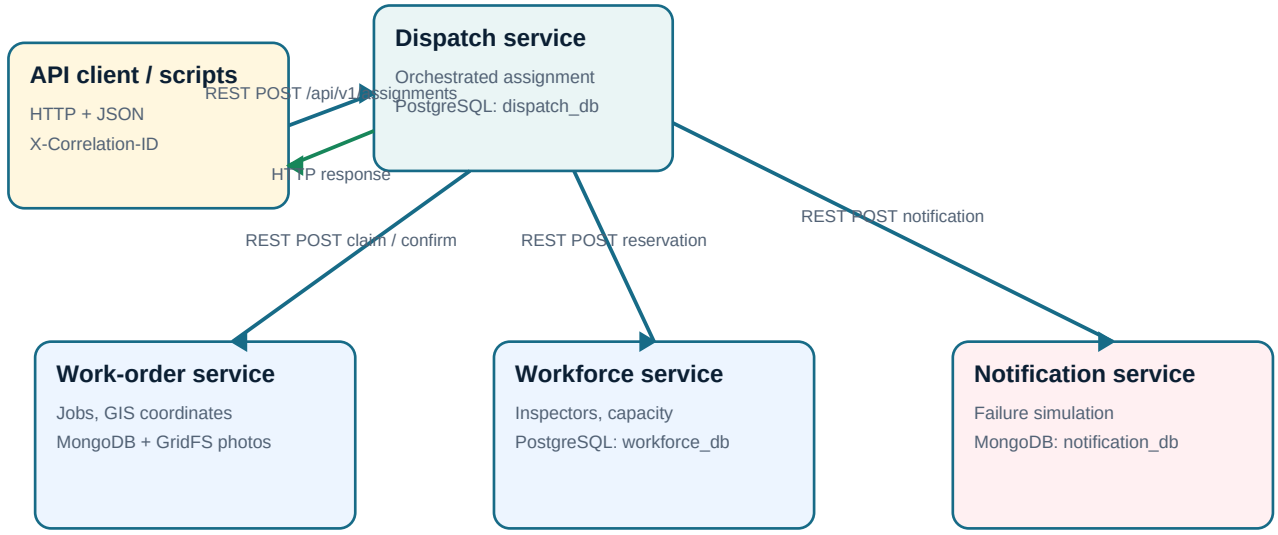
Alternatives rejected

- One shared database would simplify joins, but would violate independent ownership and allow services to bypass APIs.
- A service per table would create CRUD wrappers with chatty coupling. Work orders and photos stay together because they share one lifecycle.
- A long A-to-B-to-C-to-D chain was avoided. Dispatch calls the three capability services directly, making state and failure handling explicit.
- Kafka/RabbitMQ was deferred because Assignment 2 would require another container and Challenge 3A can demonstrate partial failure with less infrastructure.

3. Architecture

The client sends one assignment command to Dispatch. Dispatch claims the work order, reserves capacity in Workforce, confirms the work-order assignment, and requests a notification. All calls carry the correlation ID so one flow can be reconstructed from structured logs.

GeoInspect logical architecture



Isolation rule

Each service reads and writes only its own database. Cross-service references are IDs carried through REST APIs.

Deployment-ready seams: independent port, dependency file, .env.example, /health, OpenAPI, and structured logs.

Communication choice

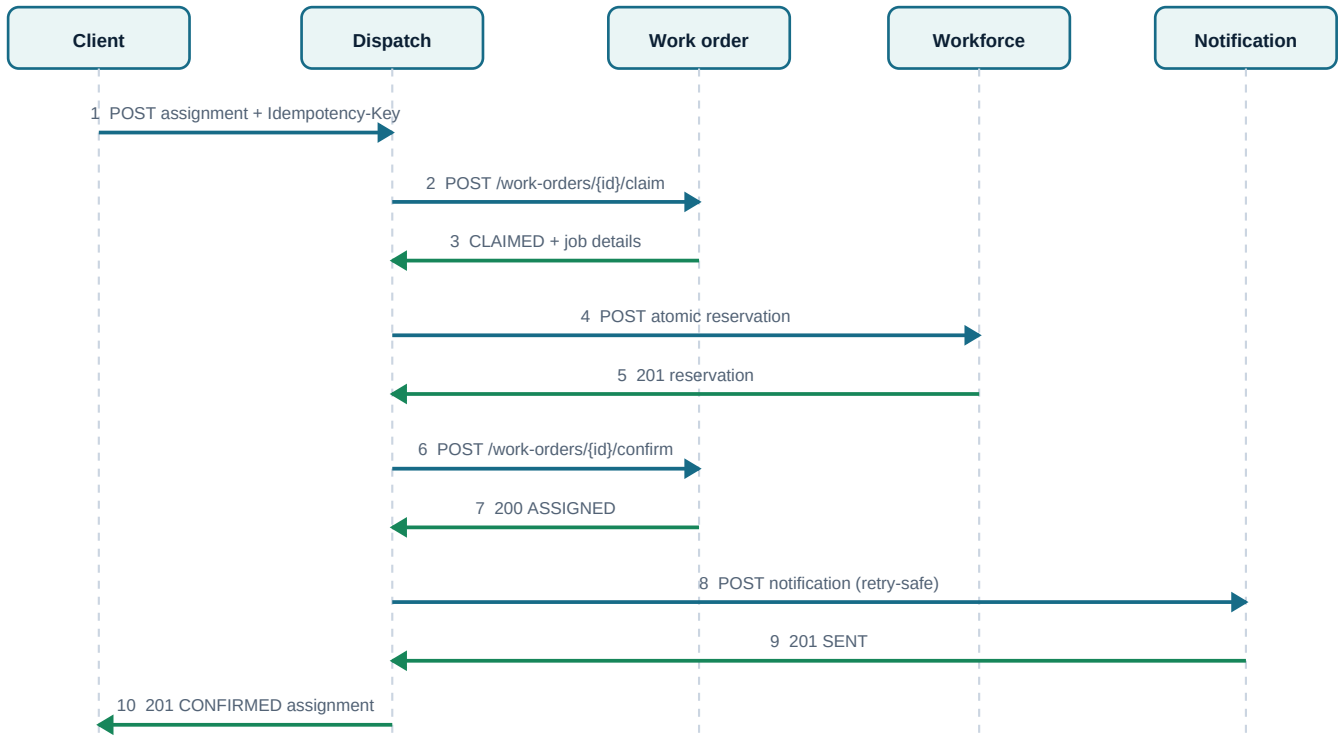
REST is used because the assignment explicitly requires HTTP APIs and because the workflow needs immediate conflict feedback. Dispatch is an orchestrator rather than a pass-through controller: it owns assignment state, idempotency, retry policy, and the visible partial-failure outcome. Database drivers communicate only with the database owned by the calling service.

Editable source

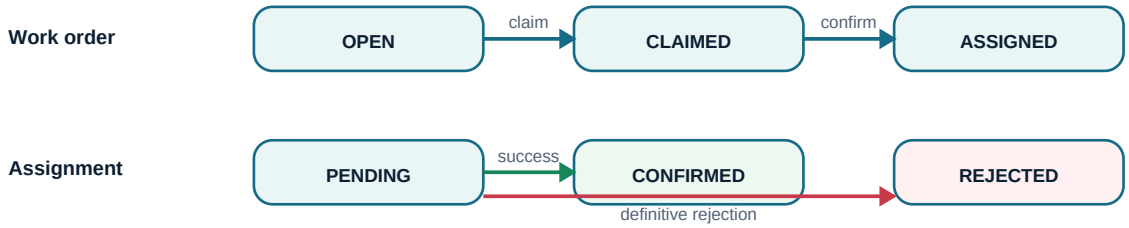
The matching component, sequence, failure, and data-ownership diagrams are provided in **docs/GeoInspect_Architecture.drawio**. Arrows name both direction and protocol for assessment and later Assignment 2 deployment planning.

3.1 Main transaction and state model

Happy path: validate the request, load the work order, reserve the inspector/slot atomically, mark the work order assigned, notify the inspector, persist the complete response, and return HTTP 201. A repeated command with the same key returns the persisted response.



Explicit workflow states

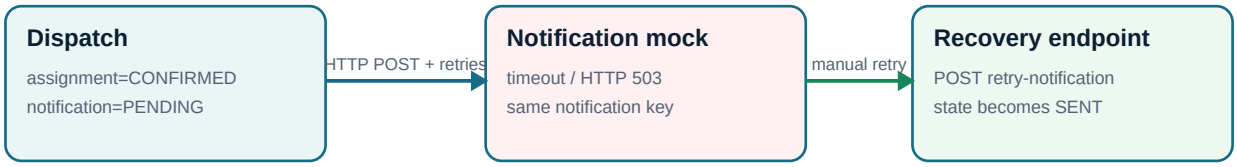


Integrity across services

There is no distributed database transaction. The design instead gives the contended reservation a strong local invariant, records the dispatch workflow state, and makes every repeated side effect idempotent. If reservation conflicts, the work order is not modified. If notification fails after the assignment is valid, the core state remains confirmed and the notification state exposes the repair action.

3.2 Failure path / 6.3 Challenge 3A resilience

Notification is the simulated slow or failing dependency. Dispatch applies a configured timeout, bounded retries, exponential backoff, and jitter. The request uses a stable logical notification key, so a provider that commits before its response is lost still produces one notification record.



Why the assignment remains confirmed

Notification is an external side effect after resource reservation and work-order assignment.

Rolling back a valid field assignment because SMS/email is unavailable would reduce availability.

Persisted PENDING state makes the partial failure visible and repeatable without creating duplicates.

Design choice and observed behavior

Forced condition	Expected behavior	Recorded result
Provider returns failure	Retries end; assignment remains CONFIRMED; notification becomes PENDING	evidence/challenge-3a-resilience/output.txt
Provider recovers	Retry endpoint sends the pending notification and marks it SENT	Same evidence file
Provider commits then delays response	Dispatch retries with the same key; provider returns the existing result	Exactly one SENT notification record

Why this failure policy

Field work should proceed when only messaging is unavailable. Persisting PENDING is a graceful, inspectable degradation and avoids undoing a valid scarce-resource reservation. The retry endpoint makes recovery demonstrable without a background worker.

Limitations

Retries run in the request path and the circuit breaker is intentionally omitted. A production version would use an outbox plus worker and automatic retry scheduling, while retaining the same notification idempotency key.

4. Data design, GIS data and photos

Each service has an independent schema and credentials. Cross-service IDs are ordinary values, not foreign keys. This prevents accidental cross-database joins and keeps each service deployable on its own.

Service	Entity / key fields	Invariant or index
Work-order (MongoDB)	work_orders: _id, asset_id, longitude, latitude, required_skill, slot, state, assignment_id	Coordinate/time validation; OPEN -> CLAIMED -> ASSIGNED
Work-order GridFS	inspection_photos.files/chunks: photo_id, work_order_id, filename, content_type	Photo metadata points to work_order_id inside same service
Workforce (PostgreSQL)	inspectors; reservations(id, inspector_id, slot, work_order_id)	UNIQUE(inspector_id, slot) prevents over-allocation
Dispatch (PostgreSQL)	assignments; idempotency(key, fingerprint, status, response)	UNIQUE(key); fingerprint detects conflicting reuse
Notification (MongoDB)	notifications: logical_key, assignment_id, status, attempts	Unique logical_key deduplicates provider retries

Why MongoDB for inspection photos

MongoDB fits flexible work-order attributes and GridFS provides streamed file storage with metadata and chunking. The REST API accepts multipart uploads and returns photo identifiers. The database choice does not mean every service shares MongoDB: the consistency-sensitive reservation and dispatch records remain in separate PostgreSQL databases.

GIS scope

The application stores WGS84 latitude/longitude and validates coordinate ranges. This is enough to make inspection orders map-ready without adding a UI or spatial server. Future versions could add a GeoJSON point with a 2dsphere index for proximity queries and PostGIS for route planning; those are outside the one-week scope.

Sample work order

```
{"asset_id":"VALVE-101","longitude":77.5946,"latitude":12.9716,"required_skill":"valve","slot":"2026-10-15T09
:00:00Z"}
```

Backup and lifecycle note

Deleting or archiving a work order should apply a same-service policy to its GridFS files. The current assignment does not implement retention or virus scanning; both belong in production hardening.

5. API design

All business endpoints are versioned under /api/v1. Every service also exposes GET /health and FastAPI-generated /docs and /openapi.json. Input models perform validation, domain exceptions map to meaningful status codes, and correlation IDs are accepted or generated and forwarded downstream.

Service	Method and path	Purpose / principal responses
Work-order	POST /api/v1/work-orders	Create job: 201; validation: 422
Work-order	POST /api/v1/work-orders/{id}/claim | /confirm | /release-claim	Idempotent participant operations: 200 / 404 / 409
Work-order	POST /api/v1/work-orders/{id}/photos	Stream photo to GridFS: 201 / 404
Workforce	POST /api/v1/inspectors	Register inspector: 201 / conflict
Workforce	POST /api/v1/reservations	Atomic slot reservation: 201 / 409
Dispatch	POST /api/v1/assignments	Main transaction; Idempotency-Key required: 201 / 409 / 422
Dispatch	POST /api/v1/assignments/{id}/retry-notification	Recover PENDING notification: 200
Notification	POST /api/v1/notifications	Idempotent mock send: 201 / 503 / timeout simulation
Notification	PUT /api/v1/mock-mode	Set normal, fail, slow, or commit-then-slow demo mode

Consistent error model

```
{"error":{"code":"SLOT_ALREADY_RESERVED","message":"Inspector is already reserved for this slot","details":{}},"correlation_id":"..."}
```

OpenAPI files

Committed specifications are in docs/openapi/work-order.json, docs/openapi/workforce.json, docs/openapi/dispatch.json, and docs/openapi/notification.json. Swagger UI is available on ports 8001-8004 at /docs while services are running.

Configuration and operability

Ports, database URLs, downstream service URLs, photo-size limit, timeouts, retry counts, and retry timing come from environment variables documented in each .env.example. Structured logs include correlation_id, service, event, and relevant entity IDs.

6.1 Engineering challenge 1 - concurrency control

Pass condition demonstrated: 20 parallel reservation attempts compete for capacity N=1; exactly one succeeds, 19 receive a meaningful conflict, and one reservation remains in PostgreSQL.

Problem

Two requests can both observe an inspector as free before either writes a reservation. A check in Python followed by INSERT is therefore a time-of-check/time-of-use race and can over-allocate the inspector.

Design

Workforce defines a database UNIQUE constraint on (inspector_id, slot). Every request attempts the INSERT. PostgreSQL serialises conflicting index writes; one transaction commits and the others raise an integrity conflict that the API maps to HTTP 409. Correctness therefore holds across processes and service replicas, not only within one Python process.

Repeatable evidence

Artifact	Claim
scripts/challenge_1_concurrency.py	Fires 20 assignment requests concurrently and queries the resulting reservation state
evidence/challenge-1-concurrency/output.txt	Shows success_count=1, conflict_count=19, reservation_count=1
services/workforce/src/data.py	Defines persistence path and handles uniqueness conflict
services/workforce/tests/test_business.py	Covers the business rejection contract

Why plain check-then-update fails

Request A reads zero reservations. Before it inserts, request B also reads zero. Both then insert. Application-level checks do not lock a shared resource and each process has independent memory. The database constraint moves the invariant to the only component that sees all concurrent writes.

Limitations

The key models exact slot equality. Overlapping time intervals would require an exclusion constraint, interval bucketing, or serialisable transaction logic. The demo intentionally uses the smallest rule that proves the required contention behavior.

6.2 Engineering challenge 2 - idempotency

Pass condition demonstrated: concurrent identical requests with one key return the same assignment and create one effect; the same key with a changed payload returns HTTP 409.

Algorithm

Step	Operation
1	Canonicalise the request payload and compute a SHA-256 fingerprint.
2	Insert an idempotency row keyed by Idempotency-Key before performing unsafe effects.
3	If the key exists, compare fingerprints. Different payload: 409. Same payload: wait/read stored outcome.
4	Persist status and the complete HTTP response so later repeats return the same result.
5	Use stable downstream identifiers so notification retries also deduplicate.

Why a fingerprint is required

A key alone cannot distinguish a legitimate replay from accidental reuse for another inspector, slot, or work order. Comparing the canonical payload protects the caller from receiving an unrelated stored response.

Repeatable evidence

Artifact	Recorded result
scripts/challenge_2_idempotency.py	Sends two same-moment calls, a sequential replay, and a changed-payload call
evidence/challenge-2-idempotency/output.txt	Both concurrent calls: 201 and same assignment_id; replay: same response; changed payload: 409
Database state in output	reservation_count=1 and notification_count=1
services/dispatch/src/data.py	Unique key plus persisted fingerprint/status/response

Concurrency behavior

The first request owns execution. A simultaneous duplicate observes the same in-progress record and resolves to the stored result instead of repeating the workflow. Failed terminal outcomes can also be replayed consistently rather than hiding a previous side effect.

Limitations

The assignment keeps idempotency records indefinitely for demo clarity. Production needs an expiry policy longer than the maximum client retry window, plus metrics for stuck IN_PROGRESS records.

7. Testing, evidence and implementation quality

Tests are layered around business invariants, then complemented by executable system scripts. The scripts use public APIs and leave database state available for inspection, which makes the evidence repeatable during the demo.

Level	Command / artifact	Result captured
Unit tests	python -m pytest	18 passed
Main flow	python scripts/end_to_end.py	Assignment CONFIRMED, notification SENT, GridFS photo uploaded and listed
Concurrency	python scripts/challenge_1_concurrency.py	1 success, 19 conflicts, one final reservation
Idempotency	python scripts/challenge_2_idempotency.py	Concurrent replay safe; mismatch rejected; one reservation and notification
Resilience	python scripts/challenge_3a_resilience.py	Failure visible as PENDING; recovery to SENT; commit-then-slow deduplicated

Fresh-clone run path

1. Create a virtual environment. 2. Install requirements-dev.txt. 3. Start PostgreSQL and MongoDB with docker compose up -d. 4. Start services with python scripts/run_all.py. 5. Run the scripts above from a second terminal. README.md lists ports, environment variables, Swagger URLs, troubleshooting, and cleanup.

Evidence index

Folder	Evidence purpose
evidence/challenge-1-concurrency	Parallel command output and explanation of database-enforced capacity
evidence/challenge-2-idempotency	First/replay/concurrent/mismatched results and single-effect database state
evidence/challenge-3a-resilience	Failure, retry recovery, and response-loss deduplication
evidence/tests	Unit and end-to-end output

Repository hygiene

Generated caches, local virtual environments, secrets, and database volumes are ignored. Each service has isolated dependencies and an environment example. The code separates API, business, and data access layers, keeping rule logic testable and persistence details replaceable.

Evidence caveat

Before final submission, rerun scripts on the final commit and capture timestamped terminal screenshots in the corresponding evidence folders. Raw text outputs are already committed and should remain the primary auditable evidence.

8. Decisions, limitations and submission appendix

Key design decisions

Decision	Options considered	Rationale and trade-off
Orchestrated REST workflow	Orchestration vs choreography	Easier to explain, trace, and return conflicts in one week. Dispatch carries coupling and must manage partial failure.
DB-enforced uniqueness	In-process lock, optimistic version, UNIQUE constraint	A UNIQUE key works across replicas and is directly demonstrable. Exact slots are less flexible than interval scheduling.
MongoDB GridFS for photos	Filesystem, object storage, byte arrays	Keeps photo metadata/streaming within work-order ownership and runs locally. Cloud object storage would scale better.
Notification failure is recoverable	Rollback assignment vs keep assignment	Messaging is secondary to a valid field assignment. PENDING state preserves work and makes repair explicit.
Challenge 3A	Time holds, saga, broker, resilience	Meets partial-failure objective with the least extra infrastructure and prepares HTTP boundaries for containerisation.

Known limitations and future work

- No authentication or authorisation; add service identities and role-based access after the assignment.
- No automatic outbox worker; add one for durable background notification retries.
- No geospatial search, routing, or offline sync; add GeoJSON indexes and a mobile client when product scope requires them.
- No automatic idempotency retention; define expiry and monitoring before production use.

Submission links and individual appendix

Item	Value to complete
GitHub repository	https://github.com/sharmaark/SS_Assignment
Google Drive / demo video	[Insert final shareable link]
Team contributions	[Add one row per student: service/challenge/report/demo responsibility and commit evidence]
Individual reflections	[Append one half-page reflection per student]

AI-use declaration

AI tools were used to assist with project planning, boilerplate, documentation structure, test-script drafting, and review. The team remains responsible for validating the design, running the code, understanding every submitted component, correcting generated material, and explaining the implementation during the demo.

Final review before upload: replace all bracketed fields, rerun evidence on the final commit, confirm the repository is public to faculty, and keep the video within 8-10 minutes.